



ABSTRAÇÃO, TIPOS E ESTRUTURAS DE DADOS

Definindo Novos Tipos

Depois de entender o que são tipos, o próximo passo é aprender a criar os nossos: estruturas próprias para representar pessoas, objetos e situações do mundo real.

AULA 11 DE 15 • JULIA + PLUTO



Estruturas de Informação

Assim como agrupamos informações relacionadas no dia a dia, podemos organizar dados que pertencem a uma mesma categoria em uma única estrutura.

Isso permite reunir elementos que fazem sentido juntos, como:

Nome e idade de uma pessoa

Marca e modelo de um carro

Definir uma nova categoria (um novo tipo) é o primeiro passo para representar, com precisão, um conceito do mundo real dentro de um programa.

EXEMPLO EM JULIA — DEFININDO UMA CATEGORIA

```
struct Pessoa
    nome::String
    idade::Int
    email::String
end
```



Representando Pessoas e Coisas do Dia a Dia



Aluno

Representa estudantes com matrícula, nome e notas.



Produto

Armazena código, descrição e preço de um item.



Carro

Contém marca, modelo e especificações técnicas.

Depois de definir uma nova categoria, podemos criar versões específicas para representar situações reais.

Por exemplo, dentro da categoria “**Pessoa**”, podemos ter **Ana**, **João** e **Maria**, cada um com suas características próprias — como idade e endereço.



Criando Instâncias na Vida Real

Depois de definir uma categoria, podemos imaginar exemplos concretos que pertencem a ela.

Categoria: Animal



Cachorro



Gato



Pássaro

Cada um pertence à mesma categoria, mas tem suas próprias características.

Instância: cada objeto criado é único, com seus próprios valores armazenados na memória.

EXEMPLO EM JULIA — TIPO E INSTÂNCIA

```
struct Carro
    marca::String
    modelo::String
    ano::Int
    velocidade::Float64
end

meu_carro = Carro("Toyota", "Corolla",
2023, 0.0)
```

Como criar uma instância de um novo tipo?

```
struct Carro
  m arca ::String
  m ode lo ::String
  ano ::Int
  velocidade ::Float64
end

m eu_carro = Carro("Toyota", "Corolla", 2023, 0.0)
```

Carro(...) → chama o construtor do tipo Carro, associando cada argumento posicional ao campo correspondente:

- m arca** → "Toyota"
- m ode lo** → "Corolla"
- ano** → 2023
- velocidade** → 0.0
- m eu_carro** → Carro("Toyota", "Corolla", 2023, 0.0) (uma instância do tipo Carro)



O Que Pode ou Não Pode Mudar

Algumas coisas podem ser alteradas depois de criadas — como um trabalho em andamento. Outras permanecem iguais depois de prontas — como uma fotografia impressa.



Coisas que não mudam

Permanecem iguais depois de criadas — como uma foto revelada ou uma escultura finalizada.

Segurança — nada é alterado sem querer

Previsibilidade — sempre do mesmo jeito

Economia — não precisam ser refeitas



Coisas que podem mudar

Podem ser alteradas sempre que necessário — como um documento em edição ou uma receita ajustada.

Flexibilidade — permitem atualizar e corrigir

Adaptação — acompanham mudanças constantes

Ao organizar informações, é importante decidir o que pode mudar e o que deve permanecer fixo.



Criando Coisas Que Podem Mudar

Às vezes precisamos representar algo que se transforma com o tempo. Criar categorias que aceitam mudanças traz mais flexibilidade e ajuda a acompanhar o que evolui na vida real.

01 Nível de água de um rio

02 Estoque de uma loja

03 Placar de um jogo

04 Saldo de uma conta

EXEMPLO EM JULIA — CRIANDO E MODIFICANDO

```
mutable struct ContaBancaria
    titular::String
    saldo::Float64
    ativa::Bool
end

conta = ContaBancaria("Maria Silva", 1000.0,
true)

# Agora podemos modificar!
conta.saldo = 1500.0
conta.ativa = false
```

Como criar e depois modificar uma instância mutável?

```
mutable struct ContaBancaria
  titular::String
  saldo::Float64
  ativa::Bool
end

conta = ContaBancaria("Maria Silva", 1000.0, true)

# Agora podem os modificar!
conta.saldo = 1500.0
conta.ativa = false
```

ContaBancaria(...) → chama o construtor, associando cada argumento posicional ao campo correspondente:

- titular** → "Maria Silva"
- saldo** → 1000.0
- ativa** → true
- conta** → ContaBancaria("Maria Silva", 1000.0, true) (uma instância mutável)
- conta.saldo = 1500.0** → sobrescreve o campo saldo – conta passa a ter saldo = 1500.0
- conta.ativa = false** → sobrescreve o campo ativa – conta passa a ter ativa = false



Síntese da Aula

Aprender a criar novos tipos — reunindo dados relacionados em uma única estrutura — nos ajuda a representar melhor pessoas, objetos e situações do mundo real, organizando as ideias e apoiando um raciocínio mais estruturado.

Tipos fixos trazem segurança e previsibilidade · **Tipos mutáveis** trazem flexibilidade e adaptação.

A SEGUIR — AULA 12

Ações Ligadas a Categorias

REFERÊNCIAS

Referências

RIBEIRO, João Araújo. Introdução à Programação e aos algoritmos. Rio de Janeiro: LTC, 2019. ISBN 978-85-216-3640-3.

FELLEISEN, Matthias; FINDLER, Robert Bruce; FLATT, Matthew; KRISHNAMURTHI, Shriram. How to Design Programs. The MIT Press, 2001.
Disponível em: www.htdp.org.

RAABE, André; ZORZO, Avelino F.; BLIKSTEIN, Paulo. Computação na Educação Básica: Fundamentos e Experiências. Porto Alegre: Penso, 2020.
Ebook. ISBN 9786581334048. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786581334048>.